Position: Web Agent Research Needs Capture-Replay Infrastructure, Not More Handcrafted Replicas

Anonymous Authors[1]

Abstract

This is a position paper. Building realistic web agent environments is extraordinarily expensive; TheAgentCompany required 3,000 person-hours for 175 tasks. This cost barrier has spawned a proprietary ecosystem where well-funded labs train on environments they cannot share, splitting the field into haves and have-nots. We argue this trajectory is unsustainable: the community should replace handcrafted website replicas with capture-replay infrastructure. A single expert demonstration, recorded on a live website, can produce a complete offline environment in minutes. We present TRACE, a proof-of-concept that captures production sites including authenticated flows, and validate scalability: 77 tasks collected in under 11 hours, a **120× reduction** in marginal time compared to replica construction. Capture-replay has real limitations; agents that deviate from recorded trajectories encounter missing responses. But these are coverage problems solvable by collecting more trajectories, not by building more replicas. We call on the community to stop investing in handcrafted environments and redirect that effort toward capture tooling, large-scale collection, and open sharing. The tools exist; the decision is whether we continue paying the replica tax or move the field forward.

1. Introduction

> *"Web agents are hill climbing in the wrong direction."*

The past two years have witnessed remarkable progress in language-model agents for web and computer use. Systems built on frontier models can now navigate complex websites, fill forms, execute multi-step workflows, and retrieve information across diverse domains (Song et al., 2025; Wei et al., 2025; Murty et al., 2024; Maia et al., 2023; Zhang et al., 2023; Li et al., 2025; Zhou et al., 2023; Garg et al., 2025; Xie et al., 2024; Xu et al., 2024; He et al., 2024; Drouin et al., 2024). Benchmarks such as Mind2Web, WebArena, WebVoyager, WorkArena, REAL, OSWorld, BEARCUBS, BrowseComp, and TheAgentCompany have provided standardized evaluation environments that enable reproducible comparisons and have driven rapid capability improvements.

Yet beneath this progress lies a troubling structural problem: **the environments we use to train and evaluate web agents bear little resemblance to the tasks that would make these agents economically valuable.** The majority of benchmark tasks fall into categories we characterize as "deep research" (trivia-style multi-hop questions that rarely require browser automation), "information seeking" (simple lookups that could often be answered by a search engine), or "atomic execution" (short, isolated actions like clicking a button or filling a single field). Long-horizon, multi-step workflows that mirror real paid work (booking complex travel itineraries, configuring enterprise SaaS tools, executing financial transactions, managing e-commerce operations) remain dramatically underrepresented. More importantly, most benchmarks are not collected from people doing paid work; they are task templates designed for evaluability rather than traces of real economic activity.

This is not an oversight. It reflects a fundamental constraint: **handcrafted environments are extraordinarily expensive to build.**

Our position: The web agent research community should replace handcrafted replicas with capture-replay infrastructure. Rather than building website replicas, we should record experts completing tasks on live websites and transform those recordings into reusable, offline environments. A single expert demonstration can produce a complete environment in minutes, compared to the hours or days required to handcraft a replica. The tools exist; the decision is whether we continue paying the replica tax or move the field forward.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Consider the costs documented in recent benchmark papers:

Table 1. Environment construction costs for prominent benchmarks.

Benchmark	Tasks	Reported Effort
WebArena	∼812	Months of development; self-hosting four functional website replicas
OSWorld	∼369	Hand-annotated tasks with initial state scripting and custom evaluators
TheAgentCompany	∼175	**3,000 person-hours** by 21 contributors over two months
REAL	∼112	Deterministic replicas of 11 real websites with custom harnesses
TRACE (ours)	**77**	**310 hours**: 300 tooling + 10.7 collection (∼8 min/task)

TheAgentCompany's transparent reporting is particularly illuminating: building 175 realistic "employee-style" tasks required 3,000 person-hours, the equivalent of 1.5 full-time engineers working for an entire year. The project involved 21 contributors including software engineers and project managers, with complex tasks requiring more than 10 hours each to design, implement, test, and verify. And these are among the most resource-rich academic teams in the field.

The implication is stark: at current construction costs, the academic community cannot build environments at the scale needed to cover the long tail of economically valuable browser tasks. We can produce hundreds of tasks, perhaps low thousands with extraordinary effort, but the space of valuable web workflows spans millions of distinct patterns across hundreds of thousands of websites.

These costs are not only financial. They consume scarce research time that could otherwise go toward agent design, failure analysis, and iteration. And because realistic environment construction is so expensive, evaluation suites remain small and often collapse to binary success metrics, which limits error analysis and makes it difficult to diagnose where, why, and how models fail.

This gap has not gone unnoticed by industry. A growing ecosystem of startups now builds browser environments for AI agent training (mec, 2025; pla, 2025; agi, 2025; kai, 2025). The business models vary: some host live environments, others deliver static datasets, and some offer both. What they share is a focus on creating replica websites and simulated workflows where AI agents can learn through reinforcement learning without triggering blocking mechanisms on real sites.

The scale of investment is remarkable. According to *The Information*, Anthropic has discussed spending more than $1 billion on RL environments over the next year (Zeff, 2025). Data-labeling giants like Scale AI, Surge, and Mercor are pivoting resources toward environment construction, with Surge reportedly generating $1.2 billion in revenue last year from AI lab contracts and recently spinning up a dedicated internal organization for RL environments. Mechanize, a startup focused exclusively on environments, offers software engineers $500,000 salaries to build them, far exceeding typical data-labeling contractor rates. Andreessen Horowitz general partner Jennifer Li summarized the landscape: "All the big AI labs are building RL environments in-house... but AI labs are also looking at third-party vendors that can create high-quality environments."

Leading AI labs treat these environments as core training infrastructure. **Browser environments have become a strategic asset: valuable, proprietary, and concentrated in the hands of well-funded organizations.**

The field is at an inflection point. Three trends are converging: (1) agents are becoming capable enough that evaluation on short, synthetic tasks no longer discriminates between systems, creating demand for long-horizon, realistic environments; (2) the cost of building such environments has created a two-tier ecosystem where frontier labs invest billions while academic groups work with hundreds of tasks; and (3) the resulting concentration raises safety concerns, as independent researchers cannot study systems trained on environments they cannot access. If the community does not change course, the gap will widen and become structural.

We believe this trajectory is problematic for the field. When the ability to train and iterate on realistic web agents depends on access to expensive proprietary infrastructure, the research community's capacity for independent investigation is constrained. Open science suffers. Reproducibility suffers. And the concentration of capability in a small number of actors raises broader concerns about the development trajectory of increasingly powerful autonomous systems. This issue affects not only academic groups, but also open-source LM companies operating on budgets that are tiny relative to frontier labs.

This paper argues for an alternative approach: capture-replay.

The core idea is simple: rather than handcrafting website replicas, we record an expert completing a task on a live website and transform that recording into a self-contained environment that can be replayed offline. A single expert demonstration (taking minutes to perform) produces a complete environment bundle including DOM states, network responses, screenshots, and interaction logs. Agents can then interact with this frozen snapshot without contacting the live web, with coverage expanding as additional trajectories are captured.

Capture-replay is not a new concept in software engineering. Record-and-replay debugging, network mocking, and browser automation testing have used similar techniques for

decades. But its application to web agent research has been surprisingly limited. We argue this represents a significant missed opportunity.

To demonstrate feasibility, we developed TRACE (Trajectory Recording and Capture Environments), an open-source capture-replay pipeline. Building robust tooling required approximately 300 hours of engineering effort: analyzing hundreds of captured sessions, developing URL canonicalization heuristics, and iterating on replay matching. But once built, the tooling amortizes: we captured six diverse environments (GitHub, Amazon, Airbnb, Kayak, Uniqlo, Ultimate Guitar) in approximately 40 minutes of total collection time, including retries and quality checks. A separate pilot study collected 71 additional tasks in 10 hours. This is roughly 8 minutes per task versus the 17+ hours per task reported for handcrafted benchmarks like TheAgentCompany, representing a **120× reduction in marginal collection time**.

We emphasize "replace" deliberately. Handcrafted replicas consume scarce human capital but are not required to achieve determinism, controlled variation, or breadth. Those properties can be achieved by capture-replay at scale through multi-trajectory collection, record-on-miss expansion, and post-processing that enables controlled perturbations. We have not yet demonstrated these techniques; our proof-of-concept validates single-trajectory capture and replay. But the economics favor building this infrastructure: even if adequate coverage requires 10 trajectories per task, that is still an order of magnitude cheaper than replica construction.

The remainder of this paper develops this argument in detail:

- **Section 2** presents a taxonomy of existing benchmarks, revealing systematic gaps in economically valuable task coverage and analyzing the cost structure of current approaches.
- **Section 3** articulates the case for capture-replay, describing its potential advantages and presenting TRACE as a proof-of-concept implementation.
- **Section 4** addresses alternative views and counterarguments, including concerns about exploration latitude, legal and ethical considerations, and technical feasibility.
- **Section 5** presents a concrete call to action with specific recommendations for researchers, benchmark creators, and the broader community.

2. The Current Landscape: Expensive Replicas, Limited Coverage

2.1. A Taxonomy of Browser Agent Benchmarks

To understand the current state of web agent environments, we systematically analyzed eleven prominent benchmarks across three dimensions: **task category** (what kind of work the agent performs), **task horizon** (how many steps or how much time tasks typically require), and **economic grounding** (whether tasks resemble work someone would pay to have completed).

Several patterns emerge from this analysis:

Pattern 1: The effort-value tradeoff. Benchmarks that emphasize economically realistic tasks (TheAgentCompany, WorkArena, REAL) require dramatically more construction effort than those emphasizing information retrieval or synthetic reasoning (BrowseComp, GAIA). This is not coincidental. Realistic execution tasks require functional environments with state that actually changes, authentication flows that work, and evaluation criteria that verify real outcomes.

Pattern 2: Horizon compression. Even benchmarks labeled as "long-horizon" often consist of relatively short interaction sequences when measured in actual steps. Mind2Web 2 explicitly analyzed this phenomenon, finding that most existing benchmarks concentrate on short-horizon tasks (Li et al., 2025). Long-horizon, multi-session workflows that characterize real knowledge work remain rare.

Pattern 3: Horizon costs explode. As agents improve, the field is pushing toward longer-horizon evaluations (e.g., METR's work on measuring ability to complete long tasks (METR, 2025)). But handcrafting long-horizon tasks scales poorly: if some tasks already require 10+ hours to design and verify, the idea of 12-hour, multi-day, or week-long workflows is untenable. Replica construction cannot keep pace with the horizon lengths we need to measure.

Pattern 4: Domain concentration. Existing benchmarks heavily favor a small set of domains: e-commerce, content management, developer tools, and travel booking appear repeatedly, while vast categories of economically important web work (financial services, healthcare portals, government services, enterprise SaaS, professional services) remain largely uncovered.

Pattern 5: The live-web evaluation problem. Benchmarks that evaluate on live websites (Mind2Web, BrowseComp, BEARCUBS, WebVoyager) face continuous validity challenges as the web changes. Those that avoid this through replicas (WebArena, REAL) gain reproducibility but lose

Table 2. Taxonomy of existing web agent benchmarks. We observe a clear pattern: benchmarks with high economic grounding (TheAgentCompany, WorkArena, parts of REAL) require the most construction effort, while scalable benchmarks (Mind2Web, BrowseComp) tend toward tasks with limited economic value.

Benchmark	Category	Horizon	Econ. Grounding	Key Limitation
GAIA	Deep research	Medium	Low	General-assistant QA; many tasks not web-specific
BEARCUBS	Info seeking	Short/Medium	Low	QA-only; no state-changing actions; small (111 tasks)
BrowseComp	Deep research	Long	Low	QA-only; short answers; open-web drift
Mind2Web 2	Deep research	Long	Low	Agentic search QA; LLM-as-judge evaluation
Mind2Web	Info seeking / Exec	Short/Medium	Medium	Live-web trajectories; action-matching eval; no deterministic env
WebVoyager	Info seeking / Exec	Medium	Low/Medium	Live websites; GPT-4V-based eval; limited domain coverage
WebArena	Execution	Medium	Medium	Replica sites; limited domains; high construction cost
REAL	Execution	Short/Medium	Medium/High	Deterministic replicas; limited task count and domains
OSWorld	Execution	Short/Medium	Medium	Desktop-focused; heavy state setup and evaluators
WorkArena	Execution	Short/Medium	High	Single platform (ServiceNow); small (33 tasks); remote-hosted
TheAgentCompany	Execution	Medium	High	Realistic workflows but extraordinarily expensive to build

coverage and realism.

2.2. The Cost Structure of Environment Construction

Why are realistic browser environments so expensive to build? The costs compound across multiple dimensions:

Website replication costs. Creating a functional replica of even a moderately complex website requires reverse-engineering its interaction model, implementing sufficient backend logic to respond meaningfully to agent actions, and maintaining consistency as the original site evolves. REAL's approach (deterministic simulations of 11 real websites) required building custom evaluation harnesses that mix programmatic checks with LLM-based judgment (Garg et al., 2025).

State initialization costs. Realistic tasks often require specific preconditions: items in a shopping cart, emails in an inbox, projects configured in a particular way. OSWorld's task suite required extensive scripting to establish correct initial states for each of its 370 tasks (Xie et al., 2024). TheAgentCompany built an entire simulated company environment with interconnected services (Xu et al., 2024).

Evaluation complexity costs. When tasks have multiple valid solutions or require judgment about partial completion, evaluation itself becomes a substantial engineering challenge. The field has increasingly turned to LLM-as-a-judge approaches, but these introduce their own costs, biases, and reproducibility concerns (Xue et al., 2025).

Maintenance costs. The web changes constantly. Benchmarks evaluated on live sites require ongoing curation to verify task validity. Replica-based benchmarks must track upstream changes if they aim to remain representative.

Credential and access costs. Many economically valuable tasks require authenticated access to real services. Benchmarks either avoid these tasks, use synthetic accounts with limited functionality, or require evaluators to provide their own credentials. Each approach constrains coverage or reproducibility.

2.3. The Concentration of Environment Access

The expense of environment construction has predictable consequences for who can build and access high-quality environments.

As discussed above, a commercial ecosystem has emerged to fill this gap. These providers offer browser environments through various models: some host live replicas, others deliver packaged datasets, and some provide both. The environments enable something impossible on real websites: running thousands of AI agents simultaneously through trial-and-error learning without being blocked.

Leading AI labs use these commercial environments for training and evaluating their web agents. As frontier labs

have exhausted most available text data for pretraining, reinforcement learning in simulated environments has become increasingly important. With per-task costs in the thousands to tens of thousands of dollars, these environments represent substantial investments, but ones that well-capitalized organizations can afford while academic groups generally cannot.

We also see this dynamic inside product companies building agents for their own products: teams are spun up to build replicas of their own UIs and workflows so they can train and evaluate at scale. This is yet another signal that handcrafted environments are expensive, bespoke infrastructure, not a sustainable research path.

These training environments are typically proprietary and contract-bound, making it hard for open-source and academic research to access or reproduce the training conditions that drive frontier results.

The result is a two-tier system: well-funded labs train agents on high-quality, realistic environments that they cannot share, while the academic community and open-source LM companies work primarily with the limited set of open benchmarks. This concentration raises concerns about:

1. **Reproducibility:** If state-of-the-art results depend on proprietary training environments, the research community cannot fully verify or build upon them.
2. **Diversity:** Concentrated environment access may lead to agents that excel on specific task distributions while failing on the broader diversity of real-world needs.
3. **Safety:** Independent safety research requires access to the same environments used for capability development; concentration of access constrains this.

3. The Case for Capture-Replay

3.1. Core Concept

Capture-replay inverts the environment construction process. Rather than building a website replica and then defining tasks on it, we:

1. **Capture:** An expert performs a real task on a live website while an instrumented browser records everything: DOM states, network traffic, user interactions, screenshots, and video.
2. **Process:** A post-processing pipeline transforms raw captures into clean, reusable artifacts: high-level action sequences (a standardized tool-call DSL), extracted credentials (for secure handling), semantic checkpoints (for partial-credit evaluation), and filtered network logs (removing analytics and non-essential traffic).
3. **Replay:** A replay engine serves the captured assets locally, allowing agents to interact with a frozen snapshot of the original session without contacting the live web.

Relation to prior record-replay systems. Record-replay is well-established in software engineering. Syscall-level tools like Mozilla rr (O'Callahan et al., 2017) enable deterministic debugging of C/C++ programs by recording and replaying system calls, but operate below the HTTP layer and cannot mock web responses. HTTP-level tools such as Polly.js (Netflix, 2018), VCR (Binié, 2011), and Nock (Azevedo, 2011) record network traffic for unit testing, using strict URL-and-method matching that fails when requests contain dynamic parameters (timestamps, session tokens, randomized IDs). Playwright's HAR replay (Microsoft, 2023) provides basic glob-pattern matching but lacks semantic understanding of request equivalence.

TRACE extends these foundations with a **multi-tier matching system** designed for the non-determinism inherent in web agent evaluation: (1) exact URL matching when possible, (2) character-frequency similarity scoring for URLs with dynamic query parameters, and (3) LLM-based disambiguation when multiple candidates remain. Crucially, TRACE tracks which HAR entries have been consumed to prevent response reuse, which is essential for flows where the same URL is requested multiple times with different expected responses (e.g., login then authenticated requests). These additions address the gap between unit-test mocking, which assumes deterministic request sequences, and agent evaluation, where exploration creates novel request orderings.

This approach offers several structural advantages:

3.2. Advantage 1: Scalability

The most significant advantage of capture-replay is speed. **A single expert demonstration produces a complete environment in the time it takes to perform the task.**

Our proof-of-concept implementation (TRACE) captured six diverse tasks across GitHub, Amazon, Airbnb, Kayak, Uniqlo, and Ultimate Guitar in about 40 minutes of total collection time including retries (about 6:40 per task). Each capture produced hundreds of HTTP responses, dozens of DOM snapshots, and complete interaction logs, artifacts that would require days or weeks to handcraft.

This changes the economics of environment construction fundamentally. At capture-replay speeds:

- A single researcher could produce hundreds of environments per week
- Crowdsourced collection becomes feasible (we built a

desktop app for non-technical collectors)

- The long tail of websites becomes accessible: any site an expert can use can become an environment

3.3. Advantage 2: Economic Grounding

Capture-replay environments are grounded in real tasks by construction. When an expert books an actual flight, configures a real SaaS tool, or completes a genuine e-commerce transaction, the resulting environment captures a workflow that someone demonstrably values enough to perform.

This contrasts with the synthetic task design required for replica-based benchmarks, where researchers must imagine what tasks would be valuable and then construct environments to support them. The imagination often falls short of reality: we design tasks we can evaluate rather than tasks that matter.

Capture-replay enables a different paradigm: **find people doing economically valuable work, record them, and build environments from their actual workflows.** This could connect web agent research more directly to real productivity gains.

3.4. Advantage 3: Democratization

Open-source capture-replay tooling can break the concentration of environment access. Any researcher with a browser can record environments from any website they can access. No proprietary infrastructure required, no licensing fees, no vendor lock-in.

This is not merely theoretical. TRACE is released as open-source software (URL redacted for double-blind review), and its captured environments are published on a public dataset host (URL redacted for double-blind review). The tooling is designed for extensibility: researchers can adapt it to their domains, contribute improvements, and build shared infrastructure without depending on commercial providers.

3.5. TRACE: A Proof of Concept

To demonstrate that capture-replay is technically feasible on modern production websites, we developed TRACE (Trajectory Recording and Capture Environments), an open-source pipeline implementing the full capture-process-replay workflow.

TRACE implements three core components: (1) a **collection** system using a stealth-configured Playwright browser that records DOM states, network traffic, user interactions, screenshots, and browser storage; (2) a **post-processing** pipeline that converts raw events to a standardized action DSL, extracts credentials for secure handling, identifies semantic checkpoints, and filters non-essential traffic; and (3) a **replay** engine that serves captured assets offline with deterministic request matching. Full implementation details are provided in Appendix A.

Demonstration dataset. We release six captured environments spanning GitHub, Amazon, Ultimate Guitar (authenticated flows with login), and Uniqlo, Kayak, Airbnb (unauthenticated). These cover e-commerce, travel search, social coding, and media sites, including sensitive interactions like payment flows. Statistics are provided in Appendix C.

The dataset is intentionally small: six tasks demonstrates *feasibility*, not comprehensive coverage. Building TRACE required approximately 300 hours of engineering effort, but once built, the six environments were captured in about 40 minutes total ($\sim$6 to 7 minutes per task). This asymmetry (high tooling cost, low marginal collection cost) is precisely why capture-replay scales where replicas do not.

3.6. Scaling Validation: A 71-Task Pilot Study

To validate that capture-replay scales beyond proof-of-concept, we conducted a pilot study using an earlier version of the TRACE tooling. A single non-expert data collector reproduced 71 tasks from the Mind2Web benchmark in approximately 10 hours of collection time (roughly 8.5 minutes per task including retries and verification).

What the pilot demonstrates. The collection time data validates our core scalability claim independently of replay quality. Even with imperfect tooling, a single collector working at modest pace ($4/hour) produced 71 task demonstrations for approximately $40 USD total. This represents a **120$\times$ reduction in per-task time** compared to TheAgentCompany's reported 17 hours per task for handcrafted environments.

What the pilot revealed. The initial captures were not immediately replayable. The pilot used an earlier data format that lacked proper HAR (HTTP Archive) recordings; the network traffic was stored in a different structure that cannot be directly replayed by TRACE's current architecture. This experience directly informed the tooling improvements that make the six demonstration environments fully functional.

Why this matters. The pilot illustrates a key property of capture-replay that replicas lack: **collection failures are cheap**. When handcrafted replica construction fails, weeks of engineering effort may be lost. When capture fails, recollection costs minutes. The 71-task pilot "failed" in the sense that those specific captures require substantial refactoring to be replayable, but the collection time investment was 10 hours, not 3,000. We can simply recollect with improved tooling.

Cost comparison. Table 3 presents a direct comparison of environment construction costs across methodologies. We use TheAgentCompany as a proxy because they transparently reported costs, a practice we commend and encourage. We acknowledge this comparison has caveats: TheAgentCompany is a desktop/computer-use benchmark (not web-only), and their 3,000 hours includes custom programmatic evaluation functions, not just environment construction. TRACE uses LLM-as-a-judge for checkpoint evaluation, which requires less per-task engineering but may differ in evaluation precision. Despite these differences, the order-of-magnitude gap in marginal collection time (17 hours vs. 8 minutes) reflects a real structural difference between handcrafted and capture-replay approaches.

Table 3. Cost comparison: handcrafted replicas vs. capture-replay. Replicas have high per-task costs; capture-replay has high fixed costs but near-zero marginal costs.

	TheAgentCompany (Replicas)	**TRACE (ours) (Capture-Replay)**
Tasks created	175	77
Fixed costs (tooling)	included	300 hrs
Collection/build time	3,000 hrs[‡]	10.7 hrs
Time per task	17.1 hrs	**0.14 hrs (8 min)**
Cost per task[†]	$855	**$0.56**
Speedup factor	n/a	**120**×
Break-even point[*]	n/a	∼35 tasks

[‡]Includes custom programmatic checkpoint evaluators; TRACE uses LLM-as-a-judge.
[†]Replica cost assumes $50/hr engineering rate; capture-replay uses $4/hr collection rate.
[*]Break-even assumes $30k tooling investment (300 hrs × $100/hr).

The 71-task pilot, despite its limitations, provides empirical grounding for the economic argument: **capture-replay marginal costs are two orders of magnitude lower than replica construction**, and this advantage compounds as collection scales.

4. Alternative Views and Counterarguments

A position paper must engage seriously with opposing views. We address the most significant counterarguments to capture-replay:

4.1. "Capture-replay limits agent exploration"

The concern: Captured environments are constrained to recorded trajectories; if an agent deviates, the replay may lack needed responses.

Our response: This concern is legitimate, and we observe it in practice. When agents deviate from recorded trajectories, they encounter requests for which no response exists in the HAR file. This is a real limitation of single-trajectory capture.

However, this is a **coverage problem, not a methodology problem**. The solution is more trajectories, not handcrafted replicas. **Multi-trajectory collection** captures multiple expert paths through the same task, providing recovery behaviors and alternatives. **Record-on-miss expansion** adds missing branches by capturing additional sessions when agents deviate. The question is not whether capture-replay can support agent exploration, but how many trajectories are needed.

Crucially, the economics still favor capture-replay. Even if a task requires 10 trajectories to achieve adequate coverage, that represents roughly 80 minutes of collection time (10×8 min), compared to 17+ hours for a single handcrafted replica. The 120× cost advantage shrinks but remains substantial. We have not yet demonstrated multi-trajectory collection at scale; this is future work. But the economic argument for investing in capture infrastructure rather than replica construction holds regardless.

4.2. "Legal and ethical concerns around captured content"

The concern: Capturing and redistributing website content raises legal (copyright, ToS) and ethical (privacy, consent) questions.

Our response: These concerns are legitimate. We advocate for: (1) credential extraction and substitution; (2) PII redaction; (3) restricted distribution with research-use agreements; and (4) community ethical guidelines (proposed in Appendix D).

Importantly, replica-based benchmarks face analogous concerns. WebArena clones Reddit's interface; REAL reproduces real website behavior. Capture-replay does not introduce novel legal risk categories; it inherits the same ambiguities that replica builders have navigated for years, but makes provenance explicit, potentially supporting clearer fair-use arguments.

4.3. "Six tasks doesn't prove scalability"

The concern: Six tasks doesn't demonstrate benchmark-scale collection or web diversity.

Our response: We separate two claims: *collection scalability* and *replay quality*. Our 71-task pilot study (Section 3.6) demonstrates collection scalability: a single non-expert collector gathered 71 tasks in 10 hours. The six fully-replayable environments demonstrate replay quality: a human following the original trajectory receives correct responses for every request, with LLM disambiguation re-

solving ambiguous matches deterministically.

Together, they show that: (1) capture scales to dozens of tasks with minimal effort; (2) the replay infrastructure correctly serves recorded responses when the trajectory is followed; (3) challenging cases (authentication, payments, dynamic content) are handled; (4) the 300-hour tooling investment built generalizable infrastructure, not task-specific code.

What we have not demonstrated: Agent evaluation at scale. As discussed in Section 4.1, agents that deviate from recorded trajectories encounter missing responses. Validating that capture-replay supports open-ended agent evaluation requires multi-trajectory collection, which we propose but have not yet implemented. The current demonstration validates infrastructure correctness via human replay; expanding coverage for agent evaluation is future work.

The pilot's captures require refactoring to match TRACE's current HAR-based format, but this illustrates rather than undermines our argument: when capture fails, recollection costs minutes. When replica construction fails, it costs weeks.

4.4. "Handcrafted replicas are necessary for controlled studies"

The concern: Research requiring controlled ablations or systematic variation needs handcrafted environments.

Our response: We disagree. Multi-trajectory collections can provide alternative paths; post-processing can enable controlled perturbations (DOM edits, resource swaps); deterministic replay provides reproducibility. We have not found research questions that strictly require handcrafted replicas.

4.5. "Commercial providers validate the need for replicas"

The concern: Companies charge $3,000 to $50,000 per task. If capture-replay sufficed, why pay these prices?

Our response: Commercial providers *do* use capture-replay methods. They sell engineering labor, not fundamentally different methodology. TRACE required $\sim$300 hours of tedious engineering (URL canonicalization, replay debugging, detection logic). Providers monetize this work; our argument is that it should be done once and shared openly.

4.6. "Capture-replay works for evaluation but not training"

The concern: RL training requires exploration and feedback from mistakes that captured environments cannot provide.

Our response: We see no fundamental barrier, only data scale challenges. Multi-trajectory capture could provide a state-action graph suitable for offline RL; checkpoints enable graded rewards; expert demonstrations support imitation learning. The offline RL literature demonstrates learning from fixed datasets is viable.

Evaluation is the more urgent bottleneck. Most academic groups cannot evaluate on realistic long-horizon tasks because environments do not exist. Capture-replay addresses this immediately.

4.7. Technical Limitations

We acknowledge several technical limitations:

Single-trajectory coverage. The current TRACE demonstration uses single-trajectory captures. We validated replay correctness via human traversal: a human following the recorded trajectory receives correct responses for all requests, with character-based matching and LLM disambiguation handling dynamic URL parameters. However, we have not validated agent evaluation, where deviation from recorded paths is common. Supporting agent exploration requires multi-trajectory collection or record-on-miss expansion, which we describe conceptually but have not implemented. This is the most significant gap between our proof-of-concept and production-ready infrastructure.

LLM-based matching non-determinism. TRACE uses LLM disambiguation when multiple responses match a request. This is a proof-of-concept convenience; production systems should default to deterministic matching via URL canonicalization and strict tie-breakers.

Anti-automation resistance. Websites deploy CAPTCHAs, bot detection, and fingerprinting. We found many high-value sites capturable with sufficient engineering, and replay is easier than capture (offline replay avoids triggering anti-bot measures). This is an ongoing arms race, but a practical obstacle rather than fundamental barrier.

Temporal validity. Captures are snapshots; a task replayed after capture may encounter mismatches as dynamic content diverges. Our validation revealed this concretely: Airbnb search (Task 6), replayed weeks after capture, required LLM disambiguation for 25% of requests because

date-dependent API calls no longer matched recorded entries (see Appendix C for full breakdown). Kayak search (Task 5), despite similar datepicker interaction, achieved 90% deterministic matching because its URLs encode dates less pervasively. Static workflows like GitHub authentication (Task 1) achieved 99.9% deterministic matching regardless of replay timing. This suggests practical mitigations: prioritize recapture for date-sensitive tasks, or implement date-aware URL normalization. Replica-based benchmarks face analogous drift (WebArena's Reddit clone does not track actual Reddit evolution), but capture-replay makes the issue explicit and measurable.

5. Call to Action

We conclude with specific recommendations:

5.1. For Research Groups and Open-Source LM Companies

1. **Redirect resources away from replica construction.** Stop building new handcrafted replicas and invest in capture-replay tooling and collection capacity.

2. **Collect domain-specific environments.** Research groups with domain expertise (finance, healthcare, enterprise software) can capture environments difficult for generalist teams.

3. **Publish captured environments with safeguards.** Share environments under research-use licenses with credential substitution and PII redaction. Document capture methodology for reproducibility.

4. **Expand capture breadth.** Use record-on-miss, multi-trajectory collection, and replay-time perturbations to expand exploration latitude without handcrafted replicas.

5.2. For Benchmark Creators

1. **Document environment construction costs.** Transparent reporting of person-hours, like TheAgentCompany's exemplary disclosure, enables informed methodology comparisons. We recommend all benchmark papers include this information.

2. **Adopt capture-replay for evaluation suites.** Replace new replica-based benchmarks with capture-replay collections to scale coverage and enable deeper error analysis.

3. **Plan migrations from replicas.** For existing replica-based suites, publish a capture-replay transition plan and a timeline for deprecating handcrafted environments.

4. **Establish shared infrastructure.** Coordinate on common tooling, formats, and distribution mechanisms for captured environments to reduce duplication and enable interoperability.

5.3. For the Broader Community

1. **Adopt and refine ethical guidelines for environment capture.** We propose comprehensive guidelines in Appendix D addressing consent frameworks, legal considerations, privacy protections, security risks, and domain-specific requirements. Community discussion should refine and formalize these norms.

2. **Create registries of captured environments.** Centralized, searchable collections would reduce duplication and enable broader access. Hugging Face and similar platforms could host these.

3. **Pressure commercial providers toward openness.** Encourage environment platforms to release subsets of their collections for research use, or to adopt open standards that reduce lock-in.

4. **Investigate legal frameworks.** Clearer understanding of copyright, ToS, and data protection implications would benefit the entire field. Legal scholarship on AI training data has relevant precedents.

5.4. Concrete Next Steps

For those convinced by our argument, we suggest immediate actions:

1. **Try TRACE.** Clone the repository, capture an environment from a website you use, replay it offline. Experience the workflow firsthand.

2. **Identify valuable tasks in your domain.** What browser workflows would genuinely save time or money if automated? These are candidates for capture.

3. **Contribute to shared infrastructure.** Submit captured environments to public registries. Report issues with capture tooling. Propose improvements to post-processing pipelines.

4. **Advocate within your organization.** If you work at a lab with access to commercial environments, advocate for releasing research subsets or contributing to open alternatives.

6. Conclusion

The web agent research community has made real progress, but handcrafted replicas are now the bottleneck. They are

slow and expensive to build, lock us into small, synthetic task sets, and collapse evaluation to binary success. As task horizons stretch to hours and days, replica construction becomes untenable, and the ecosystem splits into a two-tier system where only well-funded labs and product companies can afford realistic environments.

Capture-replay replaces that bottleneck with fast, economically grounded environments collected from real work. Deterministic replay provides reproducibility; multi-trajectory collection and branching, not yet demonstrated but economically feasible, can provide the coverage and analysis depth that replicas offer without the construction cost. The remaining challenges are about collection scale and responsible sharing, not environment engineering.

Our position, restated: The web agent research community should replace handcrafted replicas with capture-replay infrastructure. Stop building new replicas, invest in capture tooling and large-scale collection, and standardize sharing so long-horizon, economically valuable tasks become accessible to everyone. The tools exist; the decision is whether we continue paying the replica tax or move the field forward.

References

AGI Inc. https://agi.inc, 2025.

Kaizen. https://www.kaizenautomation.com, 2025.

Mechanize. https://www.mechanize.work, 2025.

Plato. https://plato.so, 2025.

Azevedo, P. T. Nock: HTTP server mocking and expectations library for Node.js. https://github.com/nock/nock, 2011.

Binié, M. VCR: Record your test suite's HTTP interactions and replay them during future test runs. https://github.com/vcr/vcr, 2011.

Drouin, A. et al. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

Garg, D. et al. REAL: Benchmarking autonomous agents on deterministic simulations of real websites. *arXiv preprint arXiv:2504.11543*, 2025.

He, H. et al. WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.

Li, Z. et al. Mind2Web 2: Evaluating agentic search with agent-as-a-judge. *arXiv preprint arXiv:2506.21506*, 2025.

Maia, M. J. A. et al. GAIA: A benchmark for general AI assistants in the wild. *arXiv preprint arXiv:2311.12983*, 2023.

METR. Measuring AI ability to complete long tasks. https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-, 2025.

Microsoft. Playwright: Mock APIs: HAR replay. https://playwright.dev/docs/mock#mocking-with-har-files, 2023.

Murty, S. et al. NNetNav: Unsupervised learning of browser agents through environment interaction in the wild. *arXiv preprint arXiv:2410.02907*, 2024.

Netflix. Polly.JS: Record, replay, and stub HTTP interactions. https://netflix.github.io/pollyjs/, 2018.

O'Callahan, R., Jones, C., Froyd, N., Huey, K., Noll, A., and Partush, N. Lightweight user-space record and replay. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*, pp. 551–564, 2017.

Song, Y. et al. BEARCUBS: A benchmark for computer-using web agents. *arXiv preprint arXiv:2503.07919*, 2025.

Wei, J. et al. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

Xie, T. et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

Xu, F. F. et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

Xue, T. et al. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*, 2025.

Zeff, M. Silicon valley bets big on 'environments' to train AI agents. *TechCrunch*, September 2025. https://techcrunch.com/2025/09/21/silicon-valley-bets-big-on-environments-to-train-

Zhang, C. et al. Mind2Web: Toward a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.

Zhou, S. et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A. TRACE Implementation Details

This appendix provides technical details on the TRACE implementation for researchers interested in understanding, extending, or reproducing the system. The complete source code is available at a URL redacted for double-blind review.

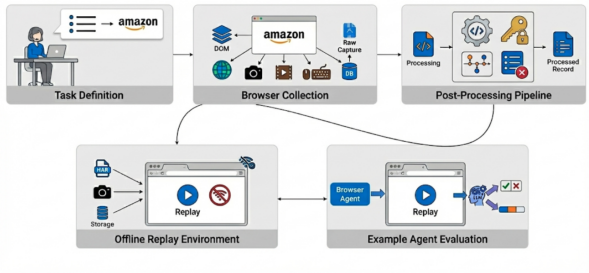

Figure 1. TRACE pipeline: collection, post-processing, offline replay, and agent evaluation.

A.1. Collection Architecture

TRACE's collection system is built on Playwright with a stealth-configured Chromium browser designed to avoid anti-automation detection while maintaining full recording fidelity.

Browser Configuration. The collector launches Chromium with carefully selected arguments to evade bot detection:

```
--disable-blink-features=
    AutomationControlled
--disable-features=VizDisplayCompositor
--no-sandbox
--disable-web-security
--enable-automation=false
```

The context is configured with realistic defaults: 1366×768 viewport, `en-US` locale, `America/New_York` timezone, and standard HTTP headers including `Accept-Language`, `Sec-Fetch-*` headers, and geolocation permissions.

Event Recording. The `Recorder` class captures events across multiple categories:

For each triggering event, the recorder captures:

1. **Accessibility Snapshot:** A YAML-formatted tree of up to 400 interactive elements extracted via Playwright's accessibility API, including element roles, names, ARIA attributes, and unique reference IDs.

2. **Screenshots:** Captured via Chrome DevTools Protocol (CDP) to avoid visual flicker, throttled to maximum one per 500ms to prevent duplicates.

3. **Network Traffic:** Full HAR (HTTP Archive) recording of all requests and responses, including headers, POST data, and response bodies (base64-encoded for binary content).

4. **Browser State:** Cookies, localStorage, sessionStorage, and IndexedDB snapshots captured at task completion.

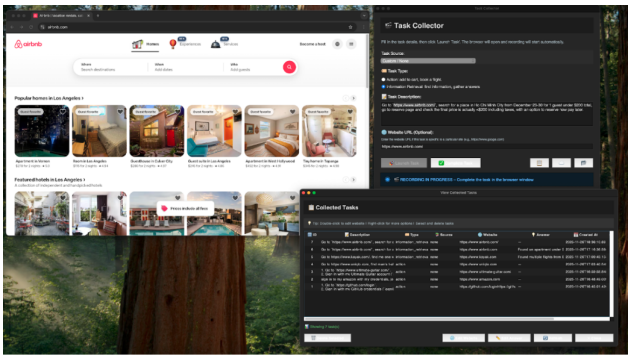

Figure 2. Collecting a task using the Tkinter desktop app.

Data Storage. Each collection produces:

- SQLite database (`tasks.db`) with task metadata and step records
- Per-step DOM snapshots in `data/doms/task_{id}/step_{n}.txt`
- Screenshots in `data/screenshots/task_{id}/*.png`
- Video recordings in `data/videos/task_{id}/*.webm`
- Capture bundle in `data/captures/task_{id}/` containing: `manifest.json` (task metadata and resource index), `recording.har` (complete HTTP archive), `storage/storage_state.json` (browser storage snapshot), and `resources/` (cached static assets)

A.2. Post-Processing Pipeline

The post-processing pipeline transforms raw captures into clean, shareable artifacts through four sequential stages.

Stage 1: Tool-Call Parsing. Raw browser events are converted to a standardized Domain-Specific Language (DSL):

The parser handles event accumulation (e.g., multiple keydown events into a single `type` action), detects navigation consequences of clicks, and preserves DOM snapshot references for each action.

Table 4. Event categories captured by TRACE.

Category	Event Types	Trigger
State:Page	load, domcontentloaded, loaded	Page lifecycle events
State:Browser	navigated, navigate_start, back	Navigation events
Action:User	click, input, contextmenu, submit	User interactions

Table 5. Tool-call DSL mapping from browser events.

DSL Action	Source Events	Parameters
`go_to`	`state:browser:navigated` (initial)	`url`
`click`	`action:user:click`, pointerdown/up	coordinates, element_info, navigates_to
`type`	`action:user:input`, keydown	value, submit, element_info
`scroll`	`action:user:scroll`	x, y (absolute coordinates)

Stage 2: Credential Extraction. An LLM-based extractor identifies credential-related interactions by analyzing:

- Tool calls with `type` actions on login-related elements
- DOM context around input fields (labels, placeholders, ARIA descriptions)
- Navigation patterns indicating authentication flows

Extracted credentials are stored in a structured format:

```
{
  "website": "amazon.com",
  "fields": [
    {"field_name": "email", "field_value":
        "user@example.com"},
    {"field_name": "password", "field_value
        ": "[REDACTED]"}
  ],
  "tool_call_ids": [3, 4]
}
```

This separation enables credential substitution during sharing while preserving the trajectory structure.

Stage 3: Checkpoint Selection. An LLM analyzes the full trajectory to identify 2 to 3 semantically meaningful intermediate states suitable for partial-credit evaluation. The prompt instructs the model to consider:

- Key actions that represent partial task completion
- Navigation events to significant pages (results, confirmation screens)
- Timestamp gaps indicating complex reasoning or significant progress

Stage 4: Ignore-List Construction. Network traffic is analyzed to identify hosts and URL patterns that can be safely omitted during replay:

- Analytics and tracking domains (Google Analytics, Facebook Pixel, etc.)
- Ad networks and retargeting services
- Non-essential third-party services (chat widgets, A/B testing)

In practice, this stage discards approximately 50% of captured HTTP requests. Modern websites include substantial tracking and analytics traffic that is entirely unnecessary for core functionality. The resulting `ignored.json` file lists URL patterns to abort during replay, significantly reducing noise and improving determinism.

A.3. Replay Engine

The replay module reconstructs captured sessions for offline evaluation through a multi-stage request matching system.

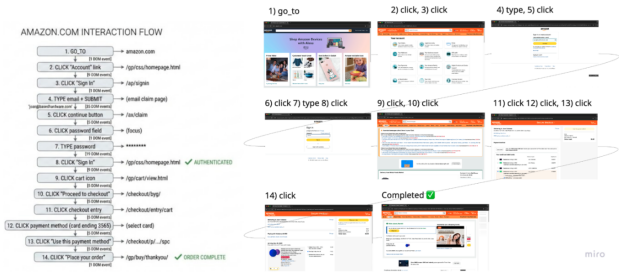

Figure 3. Offline replay and tool-call parsing for an Amazon task.

Request Routing. When the browser issues a request during replay:

1. **Ignore Check:** URLs matching patterns in `ignored.json` are immediately aborted.
2. **Exact Match:** The HAR file is searched for entries with identical method and URL base (scheme + host + path). If a single match exists, it is used directly.

3. **Character-Based Matching:** For URLs with dynamic query parameters, a character-frequency similarity score is computed:

$$\text{score} = \sum_{c} \min(\text{target\_char\_count}[c], \text{candidate\_char\_count}[c])$$

URLs are normalized before matching (removing timestamp parameters, sorting query strings). Candidates with ¿90% character overlap and matching all target characters are treated as perfect matches.

4. **LLM Disambiguation:** When multiple candidates remain after character-based filtering, the top-5 candidates (ranked by match score) are sent to an LLM for selection. The prompt provides: target request details (method, normalized URL, headers, POST data), candidate request details with response MIME types, and character match scores as additional context.

Response Fulfillment. Once a HAR entry is selected: (1) response headers are extracted, with `Set-Cookie` headers combined with newlines (per HTTP spec); (2) response bodies are decoded (base64 for binary content, UTF-8 for text); (3) the request is fulfilled via Playwright's route API with appropriate content-type headers.

Caching. LLM match decisions are cached in `matches.json` keyed by `{method}-{url}-{body_hash_16}`. Subsequent replays reuse cached decisions for determinism. The `--ignore-cache` flag forces fresh LLM matching for debugging.

Storage State. Optional `--include-storage-state` flag restores cookies and localStorage from the capture, enabling replay of authenticated sessions.

A.4. Evaluation Harness

TRACE includes a minimal evaluation runner demonstrating integration with browser-use agents:

1. **Environment Launch:** Load capture bundle, configure HAR replay routing, open starting URL.

2. **Agent Execution:** Initialize browser-use agent with task description and target website. Agent interacts with the replayed environment through standard browser automation APIs.

3. **Checkpoint Evaluation:** Compare agent's trajectory against human checkpoints using semantic similarity. An LLM judge determines whether the agent reached equivalent states, allowing alternative valid paths.

4. **Success Determination:** Binary success is assessed by comparing the agent's final state against the task's expected outcome (for action tasks) or answer (for information retrieval tasks).

B. Comparison with HTTP Record-Replay Tools

Table 6 compares TRACE's replay approach with existing HTTP record-replay tools. While these tools share the fundamental capture-replay paradigm, they differ significantly in request matching sophistication and intended use case.

Key differences. **Polly.js** (Netflix, 2018) uses adapters (fetch, XHR, Puppeteer) to intercept HTTP traffic and records to HAR format. Matching is strict by default: requests must match method, URL, headers, and body exactly. Dynamic parameters require manual configuration of "matchRequestsBy" rules to ignore specific fields. Designed for deterministic test suites where request sequences are predictable.

VCR (Binié, 2011) pioneered the "cassette" metaphor for recording HTTP interactions in Ruby. It offers configurable matchers (method, URI, host, path, body, headers) and supports regex patterns for dynamic content. However, matching remains deterministic; there is no semantic understanding of request equivalence. The "record: :new_episodes" mode can extend cassettes but does not handle request reordering.

Nock (Azevedo, 2011) provides HTTP mocking for Node.js with interceptors that match URL patterns (including regex) and request bodies. By default, interceptors are consumed after use (one-shot), which aligns with TRACE's index tracking but requires knowing the exact request sequence in advance. No fallback mechanism exists when strict matching fails.

Playwright HAR (Microsoft, 2023) offers "routeFromHAR" for replaying captured traffic. The "url" option supports glob patterns (e.g., "**/api/**") but not semantic matching. The "update" mode re-records missing requests to live servers, which is useful for extending captures but not for offline-only evaluation. No mechanism exists for disambiguating between multiple matching entries.

TRACE addresses the gap between these unit-test-oriented tools and the requirements of web agent evaluation. Agents explore unpredictably, generating request orderings that differ from the recorded trajectory. TRACE's character-frequency matching handles URL variations from timestamps and session tokens without manual configuration. LLM disambiguation resolves ambiguous cases where multiple HAR entries could satisfy a request. Index tracking ensures that requests to the same URL at different points in

Table 6. Comparison of HTTP record-replay tools. TRACE adds semantic matching capabilities designed for the non-determinism inherent in web agent evaluation.

Tool	Matching Strategy	Dynamic Params	Reuse Prevention	Primary Use Case
Polly.js	Strict URL + method	Manual config	No	JS unit testing
VCR	Configurable matchers	Regex patterns	Optional	Ruby integration tests
Nock	URL + method + body	Regex/functions	One-shot default	Node.js mocking
Playwright HAR	Strict or glob pattern	Basic wildcards	No	E2E test mocking
TRACE	Multi-tier + LLM	Auto char-similarity	Yes (index tracking)	Web agent evaluation

a session receive appropriate responses, which is critical for authentication flows where a URL may be requested before and after login with different expected responses.

C. Captured Environment Statistics

This appendix provides detailed statistics on the six demonstration environments released with TRACE. All environments are available at a URL redacted for double-blind review. Statistics are reproduced from our proof-of-concept dataset collection.

Column definitions:

- **Clicks:** Number of click events recorded
- **Tools:** Number of high-level tool calls after post-processing
- **Creds:** Whether the task involved credential entry (login flows)
- **HTTP:** Total HTTP request/response pairs in HAR file (before ignore-list filtering)
- **Cache:** Number of cached static resources
- **Shots:** Number of screenshots captured
- **DOM:** Number of DOM/accessibility snapshots captured
- **Time(s):** Task execution duration in seconds (successful run only)
- **Size:** Total capture bundle size on disk

Aggregate totals: Total HTTP requests: 7,725 (before filtering; ∼50% discarded by ignore-list); Total cached resources: 3,092; Total screenshots: 120; Total DOM snapshots: 160; Total storage: 663 MB.

Note on collection time: The Time(s) column reflects the duration of a successful task execution. Actual collection effort is approximately 6:40 per task when accounting for retries due to collection errors, website issues, or mistakes during demonstration. The six-task dataset required about 40 minutes of total collection time.

C.1. Task Descriptions

Task 1: GitHub (Authenticated). Sign in, star a repository, search for and follow a user. Exercises authentication, navigation, and account actions.

Task 2: Amazon (Authenticated). Sign in, navigate to deals, find discounted item, add to cart, proceed toward checkout. Exercises e-commerce workflow including authentication, filtering, and cart management.

Task 3: Ultimate Guitar (Authenticated). Sign in, search for tabs, interact with playback controls. Exercises authentication on media-heavy site.

Task 4: Uniqlo (Unauthenticated). Browse catalog, apply filters, view product details, add to cart. Exercises e-commerce browsing without authentication.

Task 5: Kayak (Information Retrieval). Search flights with constraints, apply filters, compare results. Exercises travel search with complex filtering.

Task 6: Airbnb (Information Retrieval). Search accommodations with filters, browse listings with map interaction. Exercises map-based search with filtering.

C.2. Observations

Several patterns emerge from the collected data:

1. **HTTP traffic scales with site complexity.** Ultimate Guitar generated the most HTTP requests (2,752), likely due to media-heavy content and third-party integrations. Travel sites (Kayak, Airbnb) had moderate traffic despite complex UIs.
2. **Bundle size correlates with cached resources.** Uniqlo and Kayak, despite different task types, both required substantial cached resources (783 and 452 respectively), resulting in larger bundle sizes.
3. **Authenticated tasks were faster.** Tasks 1 to 3 (authenticated) averaged 54.4 seconds, while tasks 5 to 6

Table 7. Complete statistics for the six captured demonstration environments.

Task	Website	Type	Clicks	Tools	Creds	HTTP	Cache	Shots	DOM	Time(s)	Size
1	github.com	Action	11	8	Yes	715	311	12	35	39.0	64 MB
2	amazon.com	Action	14	11	Yes	1,199	468	20	31	69.2	94 MB
3	ultimate-guitar.com	Action	19	15	Yes	2,752	534	20	46	54.9	117 MB
4	uniqlo.com	Action	11	10	No	1,315	783	12	12	46.1	128 MB
5	kayak.com	Info	14	11	No	816	452	32	17	110.7	154 MB
6	airbnb.com	Info	15	12	No	928	544	24	19	103.9	106 MB

(information retrieval, unauthenticated) averaged 107.3 seconds. This likely reflects the more exploratory nature of information retrieval tasks.

4. **DOM snapshot frequency varies by interaction pattern.** Ultimate Guitar (46 snapshots) and GitHub (35 snapshots) had higher snapshot counts due to more frequent triggering events (clicks, form submissions).

Even accounting for retries and mistakes, total collection effort for all six environments was approximately 40 minutes (∼6:40 per task), demonstrating the efficiency of capture-replay compared to handcrafted environment construction which can require hours or days per task.

C.3. Replay Validation Results

To validate replay correctness, we manually traversed each captured environment following the original trajectory. Table 8 reports matching outcomes across the multi-tier system.

Observations. Matching success varies substantially by site complexity. Simple authenticated flows (GitHub, Amazon) achieve near-perfect deterministic matching (99%+). Sites with heavy dynamic content (Ultimate Guitar's ad network, Uniqlo's tracking pixels) require more LLM disambiguation but still resolve correctly.

Temporal mismatch. Airbnb's high LLM requirement (25.3%) stems from temporal mismatch: the task involved date-dependent search, and replay occurred weeks after capture. Airbnb encodes search dates directly in API URLs, causing requests to diverge from recorded entries. Kayak, despite similar datepicker interaction, encodes dates differently and achieved 89.6% deterministic matching. This illustrates the temporal validity limitation discussed in Section 4: date-sensitive tasks degrade faster than static workflows.

Failed requests. The 1–2% failure rate on three tasks reflects requests for which no plausible HAR entry existed, typically dynamically-generated tracking pixels or ad callbacks absent from the original capture. These failures did not prevent task completion; the affected resources were non-essential (analytics, ads). Production systems should extend the ignore-list to cover such cases.

D. Ethical and Safety Guidelines

This appendix proposes guidelines for responsible capture and distribution of browser environments.

D.1. Legal Considerations

Copyright. Captured environments contain copyrighted material. Fair use doctrines may permit research use, but this is untested for AI training. We recommend documenting fair use rationale and limiting distribution to research contexts. Notably, replica-based benchmarks face identical concerns: WebArena reproduces Reddit's interface; REAL simulates commercial sites.

Terms of service. Most websites prohibit automated access in ToS, though enforceability varies (*hiQ v. LinkedIn*). We recommend avoiding high-volume systematic capture and respecting robots.txt.

CFAA considerations. Post-*Van Buren* (2021), mere ToS violations are not CFAA crimes, but circumventing access controls may be. Capture only from accounts you own or have authorization to use.

D.2. Privacy and Data Protection

Captures contain personal data requiring attention under GDPR/CCPA:

1. **Informed consent.** Demonstrators should consent to capture and distribution.
2. **PII detection and redaction.** Post-processing should detect and redact names, emails, addresses, financial data, and government identifiers.
3. **Third-party minimization.** Avoid capturing pages with extensive third-party PII unless necessary.
4. **Credential extraction.** TRACE separates authentication data from content; extend this to general PII.

Table 8. Replay matching breakdown for human traversal of captured environments. Deterministic matches require no LLM disambiguation; LLM-required matches were resolved correctly; failed matches had no suitable HAR entry.

Task	Website	Requests	Deterministic	LLM Required	Failed	LLM Calls	Avg Confidence
1	GitHub	725	99.9%	0.1%	0%	1	0.68
2	Amazon	676	99.1%	0.9%	0%	6	0.48
3	Ultimate Guitar	342	88.6%	11.4%	2.0%	39	0.59
4	Uniqlo	663	94.1%	5.9%	1.2%	39	0.55
5	Kayak	211	89.6%	10.4%	0%	22	0.57
6	Airbnb	501	74.7%	25.3%	1.8%	127	0.55
Total/Avg		3,118	91.3%	7.5%	1.1%	234	0.55

D.3. Security Risks

Credential exposure. Captures may contain passwords, session cookies, API keys, and tokens. Mitigations: mandatory credential extraction, secure storage, credential rotation after capture, and preference for short-lived tokens.

Vulnerability disclosure. Captures may reveal security flaws. Review captures before distribution; establish responsible disclosure procedures; withhold captures revealing serious vulnerabilities.

D.4. Misuse Potential

Capture-replay has dual-use risks. Environments could train agents for automated fraud, credential stuffing, phishing, or surveillance.

Mitigations: Access controls for sensitive domains (finance, healthcare, government); licensing that prohibits malicious applications; consider whether certain environment types (payment completion, account creation) should be captured at all; tiered access balances openness with responsibility.

Our view: Proprietary concentration does not eliminate misuse risk; it prevents independent safety research. Transparent infrastructure enables collective norm development.

D.5. Three-Tier Consent Framework

1. **Tier 1: Self-demo.** Scope: own accounts. Requirements: consent, credential extraction. Distribution: public after processing.

2. **Tier 2: Authorized.** Scope: others' demos, research accounts. Requirements: IRB review, written consent, PII redaction. Distribution: research-use license.

3. **Tier 3: Sensitive.** Scope: healthcare, finance, government. Requirements: domain compliance (HIPAA, PCI-DSS), legal review. Distribution: institutional agreements.

D.6. Domain-Specific Notes

- **Healthcare:** Never capture real patient data; treat as Tier 3
- **Financial:** Stop before payment completion; never capture real card data
- **Government:** Jurisdiction-specific regulations apply
- **Enterprise SaaS:** Obtain written authorization from system owners

D.7. Community Infrastructure

We call for: shared registries with access controls (e.g., Hugging Face), open-source compliance tooling (PII detection, credential extraction), community review for sensitive domains, and incident response procedures.

D.8. Limitations

These guidelines are not legal advice, cannot anticipate all risks, and depend on community adoption. Strict controls may recreate capability concentration. Despite limitations, explicit guidelines are preferable to the implicit norms around replicas, which sidestep these questions entirely.